

UniCNet: Unified Cycle-Accurate Simulation for Composable Chiplet Network With Modular Design-Integration Workflow

Peilin Wang¹, Mingyu Wang², *Member, IEEE*, Zhirong Ye, Tao Lu³, and Zhiyi Yu⁴, *Senior Member, IEEE*

Abstract—Composable chiplet-based architectures adopt a two-stage design flow: first chiplet design, then modular integration, while still presenting a shared-memory, single-OS system view. However, the heterogeneous and modular nature of the resulting network may introduce performance inefficiencies and functional correctness issues, calling for an advanced simulation tool. In this paper, we introduce UniCNet, a unified and cycle-accurate network simulator designed for composable chiplet-based architectures, which is open-sourced. To achieve both accurate modeling and unified simulation, UniCNet employs a *design-integration workflow* that closely aligns with the composable chiplet design flow. It supports several key features oriented towards composable chiplet scenarios and introduces the first cycle-level chiplet protocol interface model. UniCNet further supports multi-threaded simulation, achieving up to $4\times$ speedup with no cycle accuracy loss. We validate UniCNet against RTL models and demonstrate its utility via several case studies.

Index Terms—Composable chiplet architecture, interconnection network, simulator, design-integration workflow.

I. INTRODUCTION

CHIPLET-BASED architectures are promising to address the challenges of monolithic chip scaling and increased manufacturing costs [1]. Unlike traditional chiplet architectures provided by a single vendor, future composable chiplet architectures adopt a two-stage design flow [2], [3], [4]: chiplets are designed by different vendors as hard intellectual properties (IP) without full knowledge of the final integration system, and are later modularly integrated via advanced packaging technologies with essential chiplet characteristics exposed for system assembly. Furthermore, composable chiplet architectures are expected to support a single-OS software image across multiple chiplets [5], necessitating a unified system view over this two-stage design flow. However, the heterogeneous and modular nature of the resulting chiplet network leads to several design challenges. First, integrating multiple chiplets from different vendors results in a vast design space of composable chiplet network, making efficient design space exploration (DSE) difficult. Second, the modular design of chiplets can introduce integration-induced deadlock [6], [7] and demands for standard chiplet protocols [8], requiring carefully designed routing algorithms and chiplet protocol interfaces to ensure correct data transmission. To support systematic DSE while maintaining functional correctness, a unified and accurate simulator for composable chiplet networks in the pre-silicon stage is necessary [9].

Received 3 December 2025; accepted 3 January 2026. Date of publication 13 January 2026; date of current version 28 January 2026. This work was supported in part by New Generation Artificial Intelligence-National Science and Technology Major Project under Grant 2025ZD0122503, in part by the National Natural Science Foundation of China under Grant 92373103 and Grant 62204271, and in part by the Opening Project of Key Laboratory of Automotive Chip Testing and Evaluation, State Administration for Market Regulation under Grant QCP202501. (Corresponding author: Mingyu Wang.)

The authors are with the School of Microelectronics Science and Technology, Sun Yat-sen University, Zhuhai 519082, China (e-mail: wangplin@mail2.sysu.edu.cn; wangmingyu@mail.sysu.edu.cn; yezhr6@mail2.sysu.edu.cn; lutao7@mail2.sysu.edu.cn; yuzhiyi@mail.sysu.edu.cn).

Digital Object Identifier 10.1109/LCA.2026.3653809

Although several simulators for chiplet networks have been proposed, they still have some limitations in terms of accuracy, generality, and functionality. First, many simulators abstract away critical timing or microarchitectural details of the chiplet network to accelerate DSE [11], [12], which can lead to inaccurate performance predictions [14]. Second, composable chiplet networks introduce two distinct design granularities at both the chiplet and router levels, leading to several unique design features. Unified evaluation of these features is not supported on current Network-on-Chip (NoC) extended simulators, which primarily operate at the router level [15]. Third, existing simulators lack the detailed modeling of chiplet protocol interface, which is critical to capture the true dynamics of inter-chiplet interactions [14]. Table I summarizes the differences among existing simulators and our work. As a result, it is challenging to directly use existing simulators to study design optimizations for composable chiplet networks.

To this end, we develop UniCNet, a unified and cycle-accurate network simulator designed for composable chiplet architectures, which is fully open-sourced.¹ UniCNet introduces four key advancements summarized as follows:

- *Modular design-integration workflow*: It adopts a design-integration workflow that aligns with the composable chiplet architecture design flow, enabling researchers to easily perform unified design explorations.
- *Multi-chiplet oriented designs*: It supports several multi-chiplet oriented design features, including composable topology, modular routing, and heterogeneous router.
- *Chiplet protocol interface modeling*: It introduces the first cycle-level chiplet protocol interface model, which references the UCIE protocol for implementation details.
- *Multi-threaded parallel simulation support*: It enables multi-threaded simulation for large-scale chiplet networks and achieves up to $4\times$ speedup with the same accuracy.

II. UNICNET

A. Modular Design-Integration Workflow

Existing chiplet network simulators either flatten the entire network into a monolithic router-level topology [9], [10], [13], or simulate each chiplet NoC in isolation with multiple OS instances [11]. However, composable chiplet architectures follow a two-stage design flow that decomposes design concerns across both the router and chiplet levels. Unified evaluation of such chiplet-level features within a purely router-level monolithic network is challenging. Moreover, chiplet-based systems typically adopt shared-memory communication with tightly coupled interconnections [9], necessitating a single-OS, unified simulation environment.

To address these challenges, UniCNet introduces a *design-integration workflow* that closely aligns with the composable chiplet design flow and is developed based on Booksim2.0 [16]. As shown in

¹ Code is accessible at <https://github.com/wangplin/CAL-UniCNet>.

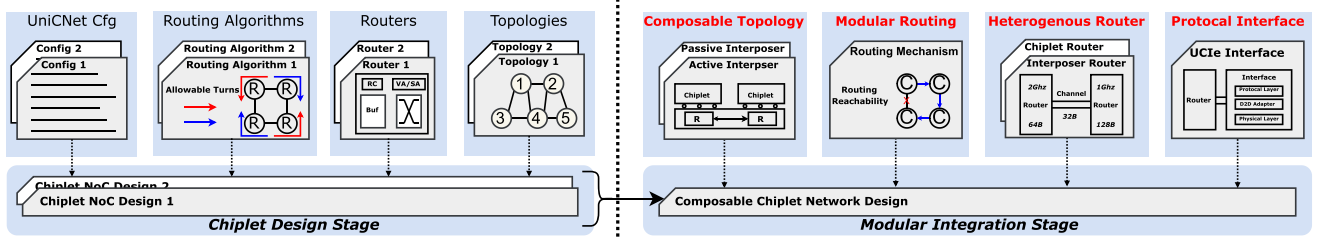


Fig. 1. The modeling strategy of UniCNet aligns with the composable chiplet design flow.

TABLE I
COMPARISON OF CHIPLET NETWORK SIMULATORS

Simulator	Accuracy		Generality*		Functionality
	C-Ap	C-Ac	Single OS	Design Level	Protocol
Garnet [10]	✓		✓	router	×
Zhi [11]	✓		×	router/chiplet	×
RapidChiplet [12]		✓	✓	chiplet	×
CNSim [9]		✓	✓	router	×
c2c-gem5 [13]	✓		✓	router	CCIX
Ours		✓	✓	router/chiplet	UCIe

*Simulators support single OS environment with two design granularities.
C-Ap: Cycle-Approximate, C-Ac: Cycle-Accurate.

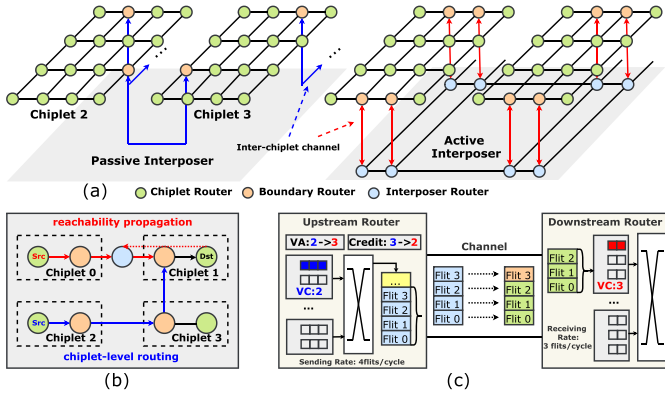


Fig. 2. Multi-chiplet oriented designs in UniCNet: (a) composable topology, (b) modular routing, (c) heterogeneous router.

Fig. 1, the composable chiplet network is constructed in two stages: (1) independent chiplet NoC design, and (2) modular chiplet integration. In the first stage, each NoC instance is configured at the router level with its own routing algorithm, router microarchitecture, and local topology. In the second stage, UniCNet composes these pre-designed chiplets into a system-level chiplet network and provides multi-chiplet oriented design features within a single unified simulation environment. These include composable chiplet topology, modular routing, heterogeneous routers, and chiplet protocol interfaces (e.g., UCIe).

B. Multi-Chiplet Oriented Designs

Composable topology construction: As shown in Fig. 2(a), UniCNet models chiplet-level topology explicitly and supports two composable chiplet organizations: *passive chiplets* and *active chiplets*, to cover different packaging technologies (e.g., 2.5D passive/active interposers). Routers in a composable chiplet network serve fundamentally different roles depending on their location. Chiplet-internal routers primarily handle intra-chiplet traffic; boundary routers connect to inter-chiplet channels with heterogeneous bandwidth and accommodate inconsistent

microarchitectural requirements; interposer routers operate in a separate fabric and are responsible for all inter-chiplet traffic. To reflect these microarchitectural and functional design differences, UniCNet defines three router types: *chiplet router*, *boundary router*, and *interposer router*.

Modular routing algorithm: The modular routing algorithm consists of distributed NoC routing algorithms, and UniCNet associates each router with the appropriate routing algorithm based on its chiplet and router type. However, these individual algorithms only ensure routing reachability within their chiplet [2]. UniCNet further implements a necessary chiplet-level routing mechanism to achieve complete routing reachability as shown in Fig. 2(b). For *active chiplet*, the destination reachability information is propagated to the interposer so as to forward packets to the correct chiplet. For *passive chiplet*, an additional chiplet-level routing algorithm is implemented.

Heterogeneous router microarchitecture: Compared with traditional NoCs that assume uniform link and homogeneous router microarchitecture, composable chiplet networks must accommodate heterogeneous bandwidth and differing microarchitectural parameters across chiplets (e.g., flow control, VC sizing) [3]. As shown in Fig. 2(c), UniCNet extends the channel model with configurable sending/receiving rates, enabling flit-level modeling of clock-domain crossing and bandwidth ratios (m flits/cycle). At the modular integration stage, UniCNet further specializes boundary routers to bridge heterogeneous microarchitectures. For example, the boundary router needs to track the inconsistent VC state of the downstream router with inconsistent VCs, which is required for VC allocation and flow control.

C. Chiplet Protocol Interface Modeling

Composable chiplet architectures rely on standardized chiplet protocols to achieve multi-vendor interoperability [5]. However, none of the prior works model it systematically. As shown in Fig. 3, UniCNet introduces a chiplet protocol interface model to support the modeling of mainstream chiplet protocols and references the *Universal Chiplet Interconnect Express (UCIe)* protocol for implementation details [8].

Protocol packet (un)packing: In the protocol layer, the incoming protocol flit would be (un)packed to the UCIe flit or transmitted directly to the adapter according to the protocol flit size. Since the flow control of data flit is implemented between the protocol layer in both upstream and downstream controllers, the (un)packing number of flit needs to be recorded and used for later received credit (de)serialization.

Adapter retrying mechanism: The overview of adapter retrying mechanism is illustrated in Fig. 3. ❶ When a UCIe flit is received from the protocol layer, the upstream adapter will increment its sequence number and transmit it to the physical layer if the retry buffer has capacity. ❷ A copy of flit is queued in the retry buffer for potential retransmission. ❸ The downstream adapter will verify each received flit according to the configured error rate, and transmit an ACK/NAK flit which contains the sequence number of the last correctly received flit.

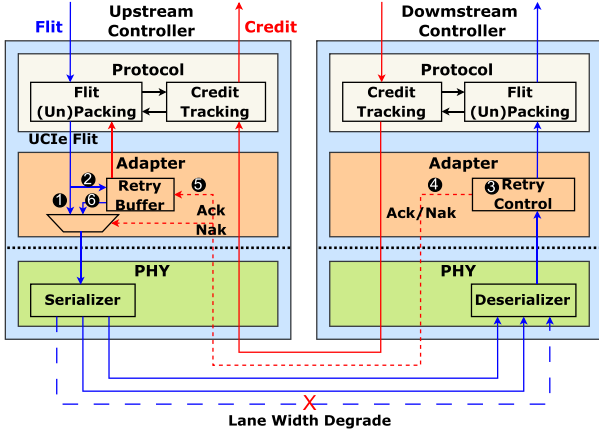


Fig. 3. Chiplet protocol interface model of UCie in UniCNet.

④ If the received flit is correctly verified, an *ACK* flit will be sent back and update the last correctly received sequence number. Otherwise, the downstream adapter will: (1) drop this wrong flit, (2) send a *NAK* flit, (3) discard all subsequent received flits, and (4) halt future *ACK/NAK* flit generation until the retry flit passes the verification. ⑤ When the upstream adapter receives an *ACK* flit, it will pop the copy of correctly received flit from the retry buffer according to the carried sequence number. ⑥ Otherwise, it will prioritize handling the *NAK* flit and resend all flits with the *Go-Back-N* mechanism.

PHY electrical behavior: UniCNet models PHY electrical behavior through SerDes and lane-width degradation. For SerDes modeling, a flit is serialized into multiple narrower flits according to the configured SerDes ratio, and these serialized flits are sent sequentially over the downstream physical layer. For lane-width degradation modeling, under a degraded heterogeneous-channel configuration, not all serialized flits can be transmitted in a single cycle. In this case, the deserializer requires additional cycles to receive all serialized flits before deserializing back to the original flit.

D. Multi-Threaded Parallel Simulation Support

Composable chiplet networks often scale to hundreds or even thousands of routers [15], making single-threaded cycle-accurate simulation impractical due to the massive number of events. Moreover, strict inter-router state dependencies and serial processing of multiple channel-packet pairs within a router severely limit exploitable parallelism in such simulations. Existing chiplet network simulator abstracts detailed circuit-level allocation logic and makes explicit trade-offs between simulation accuracy and multi-threaded implementation [9].

Decouple and parallelize evaluate stage: UniCNet exploits the FIFO nature of channels and routers/protocol interfaces to reorganize per-cycle processing into three logically separated stages: *ReadInput*, *Evaluate*, and *WriteOutput*. The intermediate *Evaluate* stage updates only local state based on the data read in *ReadInput*, and all effects on channels are deferred to *WriteOutput*; this enables parallel evaluation of different instances without violating cycle-accurate semantics. Moreover, profiling shows that the *Evaluate* stage dominates the overall simulation time (over 70%) [9]. UniCNet preserves accurate circuit-level behavior, and instead parallelizes the most time-consuming *Evaluate* stage using multi-threaded execution. To balance the parallel workload of different instances (e.g., distinct routers and interfaces) under various configurations, UniCNet further supports dynamically scheduling them onto available threads.

TABLE II
DETAILED CONFIGURATIONS USED FOR VALIDATION

Network	Passive/Active, four 4×4 2D Meshes, C2C latency: 9/4 cycles [17]
Router [18]	4-stage, 4 VCs, 1 packet-sized (4 flits) buffer, modular XY routing
Simulation	Uniform traffic pattern, simulated through Synopsys VCS
Interface	Packet/UCie: 1/1 flit, Protocol/Adapter/PHY+C2C: 1/3/4 cycles

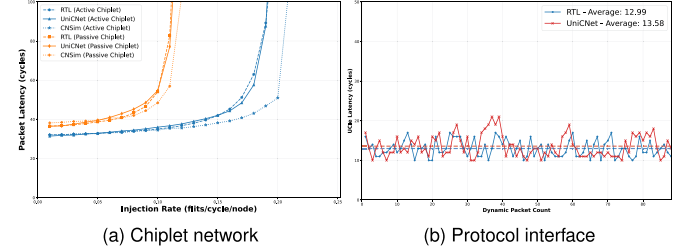


Fig. 4. The results of validation studies.

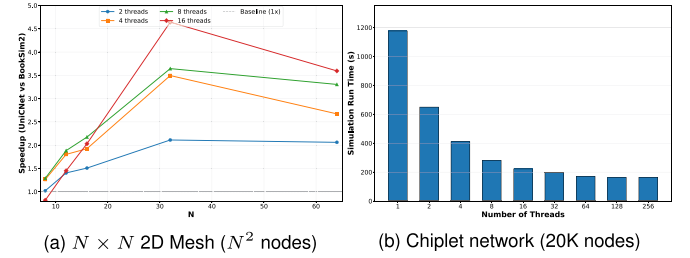


Fig. 5. Speedup of parallel simulations for UniCNet.

III. VALIDATION AND CASE STUDIES

A. Validation Studies

We validate the UniCNet in terms of chiplet network and protocol interface. For chiplet network, we take *passive/active chiplet* networks shown in Fig. 2(a) as baseline and compare the results with RTL model and CNSim [9]. For protocol interface, we validate the controller part of chiplet protocol interface model against UCie controller RTL. Table II summarizes the detailed configurations used to evaluate UniCNet.

Fig. 4(a) shows the load-latency curve for both networks. The results show that the difference in zero-load latency and throughput between UniCNet and RTL models is within 5%, while for CNSim, the difference is up to 10.27%. Fig. 4(b) shows the latency for each packet passing through the UCie interface. During periods of either sharp latency increase or stable fluctuations, the latency trend of UniCNet follows that of the RTL model. The obtained average packet latency of UniCNet is 13.58 cycles, with a less than 4.54% deviation compared with the RTL model (12.99 cycles). Both the alignment of the latency trend and the small deviation indicate the effectiveness of UniCNet.

B. Performance Studies

Fig. 5(a) shows the simulation speedup over Booksim2.0 for $N \times N$ 2D Mesh. The maximum speedup is from $1.29 \times$ to $4.64 \times$ on AMD Ryzen 7745HX CPU, and an average speedup of $2.2 \times$ is achieved across all configurations. Fig. 5(b) shows the simulation times for a *active chiplet* network with 32×32 integrated 4×4 NoCs and 64×64 interposer. The 4-thread and 32-thread UniCNet on AMD EPYC 9554 CPU are about $2.87 \times$ and $5.94 \times$ faster than the single-thread simulation.

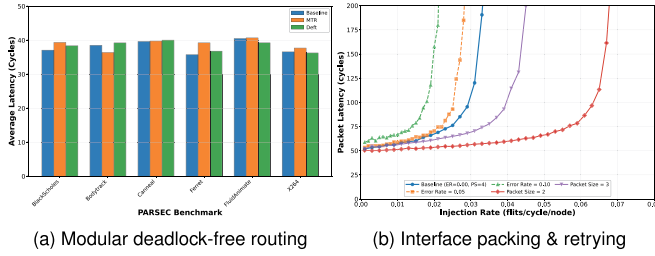


Fig. 6. The results of case studies.

C. Case Studies

Modular deadlock-free routing: We evaluate the end-to-end performance of modular deadlock-free routing algorithms under PARSEC benchmark: MTR [2] and Deft [6]. Specifically, we integrate UniCNet with Gem5 [19] simulator and adopts *active chiplet* configuration. Fig. 6(a) shows the average latency comparison. Both deadlock avoidance schemes effectively prevent infrequent integration-induced deadlocks but still incur moderate performance loss compared with ideal routing (baseline). This case study demonstrates UniCNet’s capability for optimizing modular routing strategies.

Interface packing & retrying mechanism: We evaluate how protocol packing efficiency and adapter retrying behavior impact composable chiplet-network performance. Fig. 6(b) shows the results for different error rates (ER) and UCIE packet sizes (PS) around a baseline (packet/UCIE: 4 flits). Under the *Go-Back-N* retry scheme, increasing the error rate to 0.10 raises zero-load latency by about 19% and reduces the saturation injection rate by 34.4%. For packing efficiency, fine-grained packetization with 2 flits achieves $2.1\times$ higher saturation throughput with comparable latency. These results show that flit error rate and protocol packetization have a significant impact on chiplet network performance, and illustrate UniCNet’s ability to capture dynamic inter-chiplet interactions.

IV. CONCLUSION

In this paper, we present UniCNet, a simulator designed to evaluate networks for composable chiplet architectures. UniCNet adopts a design-integration workflow that mirrors the two-stage chiplet design flow, which achieves both modeling accuracy and unified evaluation. With its modular design, UniCNet supports multiple key designs oriented towards multi-chiplet scenarios, including composable topology, modular routing, and heterogeneous router. Moreover, UniCNet introduces the first cycle-level chiplet protocol interface model referenced from UCIE and verifies it against the RTL model. For large-scale chiplet networks, UniCNet is optimized to enable multi-threaded simulation,

achieving up to $4\times$ speedup compared with Booksim2.0. These advantages underscore the potential of UniCNet to facilitate the design space exploration in composable network design and full-system evaluation.

REFERENCES

- [1] S. Naffziger et al., “Pioneering chiplet technology and design for the AMD EPYC and Ryzen processor families: Industrial product,” in *Proc. Int. Symp. Comput. Archit.*, 2021, pp. 57–70.
- [2] J. Yin et al., “Modular routing design for chiplet-based systems,” in *Proc. Int. Symp. Comput. Archit.*, 2018, pp. 726–738.
- [3] S. Bharadwaj and T. Krishna, “InC2: Design of interconnection systems for composable chiplet architectures,” in *Proc. IEEE/ACM Int. Workshop Netw. Chip Archit.*, 2024, pp. 1–6.
- [4] G. S. Kalsi et al., “BiFrost: A composable, resilient interconnect network architecture for scalable AI systems,” *IEEE Micro*, vol. 45, no. 5, pp. 67–78, Sep./Oct. 2025.
- [5] Arm chiplet system architecture, Jan. 2025. [Online]. Available: <https://developer.arm.com/documentation/den0145/latest>
- [6] E. Taheri, S. Pasricha, and M. Nikdast, “DeFT: A deadlock-free and fault-tolerant routing algorithm for 2.5D chiplet networks,” in *Proc. Des. Autom. Test Eur. Conf.*, 2022, pp. 1047–1052.
- [7] D. Yu et al., “VDA: A simple but efficient virtual-channel-based deadlock avoidance scheme for scalable chiplet networks,” in *Proc. Great Lakes Symp. VLSI*, 2024, pp. 357–363.
- [8] Universal chiplet interconnect express (UCIE) specification rev 1.0, Feb. 2022. [Online]. Available: <https://www.uciexpress.org>
- [9] Y. Feng et al., “Evaluating chiplet-based large-scale interconnection networks via cycle-accurate packet-parallel simulation,” in *Proc. USENIX Annu. Tech. Conf.*, 2024, pp. 731–747.
- [10] S. Bharadwaj, J. Yin, B. Beckmann, and T. Krishna, “Kite: A family of heterogeneous interposer topologies enabled via accurate interconnect modeling,” in *Proc. ACM/IEEE Des. Autom. Conf.*, 2020, pp. 1–6.
- [11] H. Zhi et al., “A methodology for simulating multi-chiplet systems using open-source simulators,” in *Proc. Int. Conf. Nano. Comput. Commun.*, 2021, Art. no. 18.
- [12] P. Iff et al., “RapidChiplet: A toolchain for rapid design space exploration of inter-chiplet interconnects,” in *Proc. Int. Conf. Comput. Front.*, 2025, pp. 168–171.
- [13] L. B. Alvarez, G. Chehaibar, S. Busch, P. Benoit, and D. Novo, “c2c-gem5: Full system simulation of cache-coherent chip-to-chip interconnects,” in *Proc. Des. Autom. Test Eur. Conf.*, 2025, pp. 1–7.
- [14] S. Chen et al., “The survey of 2.5D integrated architecture: An EDA perspective,” in *Proc. Asia South Paci. Des. Autom. Conf.*, 2025, pp. 285–293.
- [15] Z. Yang et al., “Challenges and opportunities to enable large-scale computing via heterogeneous chiplets,” in *Proc. Asia South Pacific Des. Autom. Conf.*, 2024, pp. 765–770.
- [16] N. Jiang et al., “A detailed and flexible cycle-accurate network-on-chip simulator,” in *Proc. Int. Symp. Perform. Anal. Syst. Softw.*, 2013, pp. 86–96.
- [17] D. Stow, Y. Xie, T. Siddiqua, and G. H. Loh, “Cost-effective design of scalable high-performance systems using active and passive interposers,” in *Proc. Int. Conf. Comput. Aid. Des.*, 2017, pp. 728–735.
- [18] D. U. Becker, “Efficient microarchitecture for network-on-chip routers,” Ph.D. Dissertation, Stanford University, 2012.
- [19] N. Binkert et al., “The gem5 simulator,” *SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, Aug. 2011.